

Agenda

- ① Access Modifiers
- ② Constructors & its types
- *** ③ Memory management in Java
- ④ Pass by value vs Pass by reference

1] Access Modifiers

Access modifiers defines how accessible is a member of a class.

- ↳ attributes
- ↳ behaviour

```
class Student {  
    access-modifier    int data-type    age; attr-name;  
    access-modifier    return-data-type  
    method-name ( .... ) {
```

```
    }  
}
```

public void calculate (int x) {
 :
}

```
class Student {  
    → int age;
```

```
}
```

4 access modifiers in Java:

① private

② default

③ protected

④ public

```
main () {
```

```
Student s = new Student();
```

```
s.age = -1;
```

```
}
```

```
eg: class Student {  
    public String name;  
    private int age;  
    default → String email;  
    protected double psp;  
  
    public void changeBatch(){  
        :  
    }  
  
    private boolean pauseCourse() {  
        :  
    }  
}
```

① private : then that member is only accessible within the class. No other code outside the class can access that member.

② default:

To use this, don't write anything before the member declaration.

Any class within package can access this member.

↳ folder of

java files

private → ~~some~~ class

default → same package

protected → same package + child in diff package

package-1

package-2

Student

private void pauseCourse()

void changeBatch()

protected void giveMock()

IntelligentStudent
extends Student {

}

Super Intelligent
Student {

③ protected

3

Any member which is protected can be accessed within the package + it can also be accessed via its child class in a different package.

④ public

Most lenient access modifier. A public member can be accessed from anywhere.

Access modifier	same class	diff class same package	diff class diff package	diff class diff package
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Constructors

Class $\rightarrow$ Blueprint of an entity

Object $\rightarrow$ instance of class

int a; $\leftarrow$ Definition

a = 2; $\leftarrow$ Initialization

int a = 2 $\leftarrow$ Declaration &
Initialization

A class has multiple attributes which needs to be initialized when an object is created.

Main.java

```
Student st = new Student();
```

```
st.name = Bhavik;
```

```
st.age = 26;
```

```
st.psp = 34.0f;
```

has no return type

A **constructor** is a **special method** used to initialize attributes of a class.

Types: ① Default

② No-args

③ Parameterized

④ Copy constructors

class Student {

int age;

String name;

Constructor



public Student () {

..
..
..

}

3

9

① Default constructor:

This constructor is given by Java to us when we have not defined our own constructor.

```

                                Student o =
                                new Student();

Class Student {
    int age;
    public String name;
    public Student () {
        age = 0 ;
        name = null;
    }
    void course Pause () {
    }
}
```

In a default constructor all the attributes of a class are set to their default values as per their data types.

Data type	default value
int	0
float / double	0.0
boolean	false
char	"
String	null
Custom class	null

② No-args constructor

This is a custom constructor that is added by the developer.

As per the name this constructor doesn't take any arguments. It is mostly used to initialize an object with custom default values.

Note: Whenever you create a custom constructor, java compiler won't give you a default constructor.

```
class Student {
```

```
    int age;  
    String name;
```

Constructor
by dev



```
    public Student () {
```

```
        this.age = 20;
```

```
        this.name = "ABC";
```

```
    }
```

```
}
```

The only similarity b/w default & no-args constructor is that object creation code looks the same for both.

```
Student st = new Student();
```

③ Parameterized constructors

These constructors can take multiple parameters.

```
Class Student {  
    int age;  
    public String name;  
    public Student (String name, int age) {  
        Refers to → this.name = name;  
        refers    this.age = age;  
    }  
}
```

3

}

```
Student st = new Student ("Bhavik",  
                           24);
```

```
Student st1 = new Student ();
```



Will this work? CTE

Rules about constructors:

- ① 1 class can have multiple constructors.
- ② Parameterized constructors need to have unique signature

Signature of a constructor is made up by list of datatypes of arguments that the constructor is accepting.

```
public Student (String name, String university)  
↳ (String, String)
```

```
public Student (String name, int age)  
↳ (String, int)
```

multiple
Can a constructor like these be created in a class?

```
(String, String)
```

1. public Student (String name, String university)
public Student (String name, int age)
✓ (String, int)

2. public Student (String name, int age)
public Student (int age, String name)
✓ (int, String)

new Student ("ABC", 20);
new Student (20, "XYZ") ;

3. public Student (String name, String email)
public Student (String name, String university)
✓ (String, String)

Student st = new Student ("Bhavik",
" @.com");
= new Student ("Bhavik",
" Mumbai Uni ");

③ Constructors name should be class name.

④ Constructors can have access modifiers:

→ private constructor : Only class is allowed to create an object

→ default constructor : Only classes in your package can create obj

→ protected constructor : Only classes in your package or your child classes can create your obj

→ public constructor : Any 1 can create your object.

3] Memory management in Java

There are two main important memory when some code is getting executed.

① Stack : Function stack

② Heap : Not DS based heap. Just a collection of objects.

```
Class Student {  
    String name;  
    int age;
```

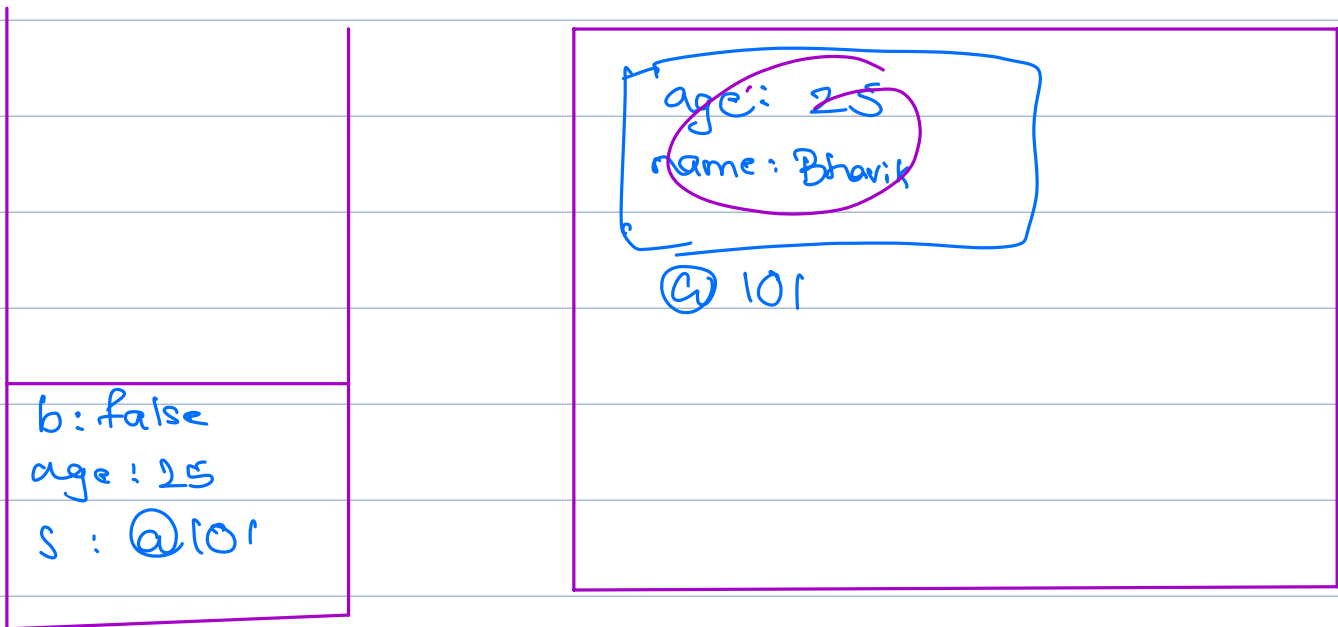
```
    public Student (int age,  
                    String name) {  
        this.name = name;  
        this.age = age;  
    }
```

```

Main C { boolean b = false
    int age = 25;
    Student s = new Student (age,
                             "Bhavik");
    sout (s.name);
}

```

3



Primitives are stored directly in the stack.

(with their attribute values)

Objects are stored in heap & their

address is stored in stack.